Atividade: Reconhecimento de Pokémon em Vídeo com Deep Learning

Machine Learning e Deep Learning - Profa. Thais Amanda Santos
Andrew Bertelli, Bernardo Kuster Ragugnetti, Eduardo Sochodolak e Pedro Henrique Moreira - 10/11/2026

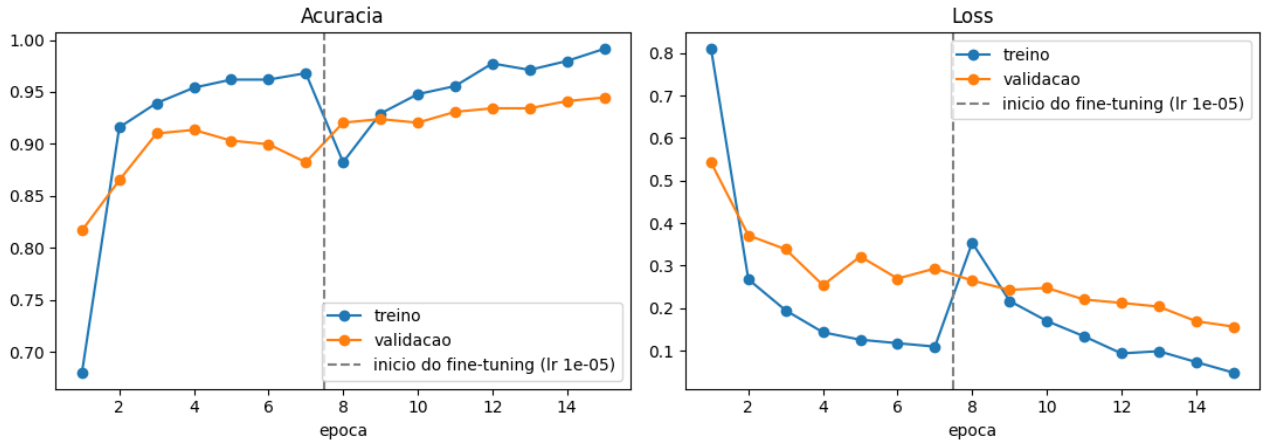
Escolhemos Pikachu, Psyduck e Charmander, além da classe fundo. Pikachu e Psyduck são os dois amarelos, como o PDF sugere. No lugar de objetos físicos usamos os modelos 3D da Pokemon-3D-api, com autorização da professora. O script `gerador/gerar_videos.py` monta uma sala 3D, coloca o Pokémon no centro e grava um vídeo com a câmera passando por frente, lados, costas, de cima, de baixo, perto e longe. Cada classe tem 6 vídeos de 20 segundos, 3 em sala clara e 3 em sala escura, e em cada vídeo a cor das paredes, a luz, os objetos espalhados e o ponto de partida da câmera são sorteados. O `extrair_frames.py` tirou de 347 a 413 fotos por classe. O vídeo de teste foi gravado numa sala com parede de tijolo e chão xadrez, um padrão que não aparece no treino.

Treino e vídeo de teste

O modelo final acertou 94,46% das fotos de validação. A validação ficou com o vídeo v04_mix_escuro de cada classe, porque o sorteio por vídeo do `train.py` usa a mesma semente em todas as classes. O pior caso é o Pikachu, com recall de 75,8%: 7 frames dele viraram Psyduck e 6 viraram fundo.

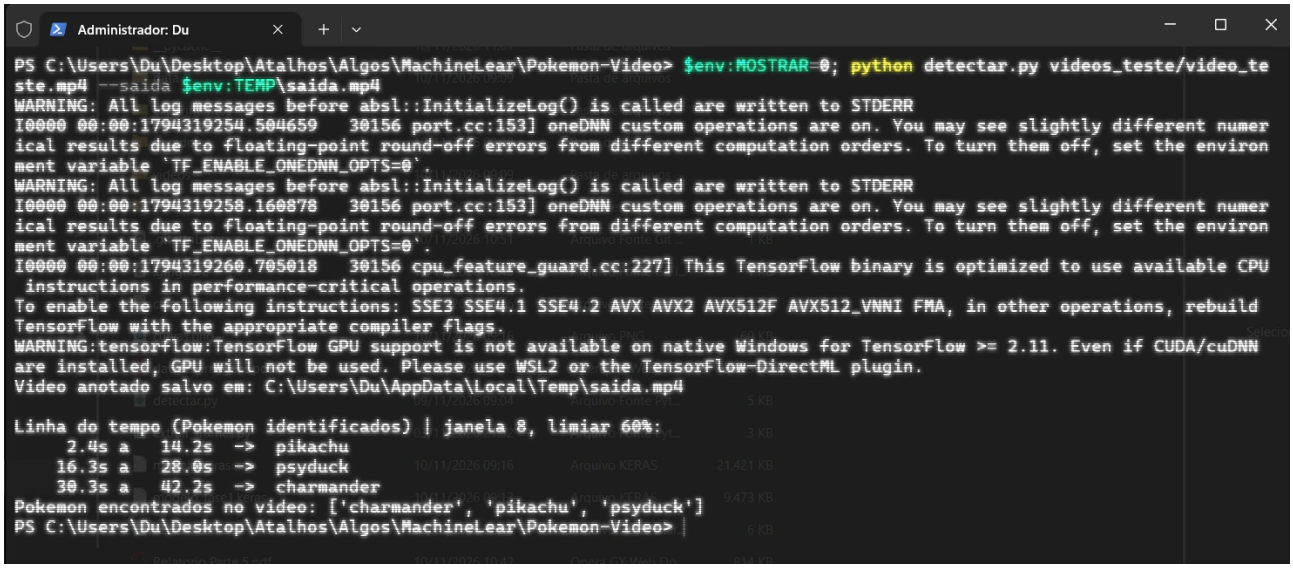


Print do terminal: matriz de confusão no fim da fase 1 e depois do fine-tuning.

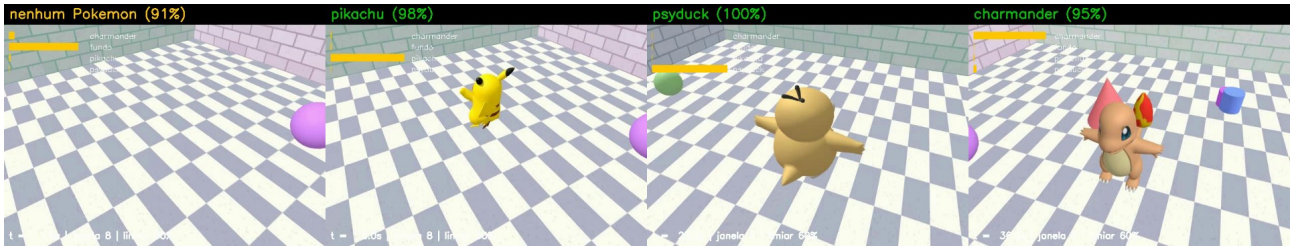


curvas.png do treino final: fase 1 (épocas 1 a 7) e fine-tuning com lr 1e-5 (épocas 8 a 15).

No vídeo de teste, os três Pokémon foram encontrados nos trechos certos (eles aparecem de 2 a 14 s, de 16 a 28 s e de 30 a 42 s). Olhando frame por frame, o modelo acertou 99,6% dos frames. Com a suavização (janela de 8 frames e limiar de 60%) o acerto fica em 94,9%, porque nas trocas de Pokémon o rótulo demora alguns frames para mudar e mostra ?.



Print do terminal: linha do tempo do vídeo de teste.



saida_video_teste.mp4 em 1 s, 8 s, 22 s e 36 s.

1. Por que separar treino e validação por vídeo?

Treinamos de novo com `--split foto`, que sorteia 20% das fotos de todos os vídeos. A acurácia de validação subiu para 99,68%, contra 94,46% do split por vídeo. Mas no vídeo de teste o modelo do split por foto acertou 98,5% dos frames, contra 99,6% do split por vídeo. Ou seja, o modelo não ficou melhor, só a validação ficou mais otimista.

Isso é vazamento de dados. A 4 fotos por segundo, um frame e o seguinte são quase a mesma imagem, então no split por foto quase toda foto de validação tem uma gêmea no treino, e o modelo é avaliado em algo que já viu. A curva mostra o sinal que o PDF descreve: no split por foto a validação já começa em

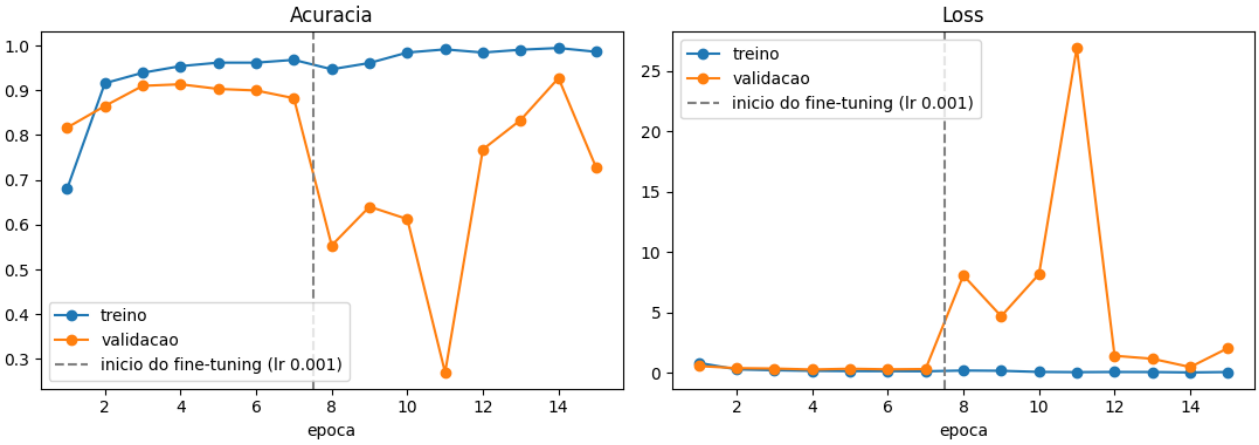
93,7% na primeira época, contra 81,7% no split por vídeo. O número que representa o desempenho real é o do split por vídeo, porque mede o modelo em vídeos inteiros que ele nunca viu, que é a situação do vídeo de teste. Mesmo ele ainda é um pouco otimista no nosso caso, porque treino, validação e teste saem do mesmo gerador 3D.

2. O fine-tuning ajudou?

Sim. No fim da fase 1 (só a cabeça), o melhor resultado foi na época 4: 91,35% de acurácia e loss de validação de 0,254. Depois de 8 épocas de fine-tuning das últimas 30 camadas com taxa 1e-5, a validação foi para 94,46% e a loss caiu para 0,157, descendo em quase todas as épocas. Na matriz, a maior diferença foi no Pikachu: na fase 1, 14 frames dele viravam Psyduck e o recall era 68,2%; depois do fine-tuning foram 7 frames e 75,8%. O recall do fundo passou de 96,2% para 100%. Faz sentido, porque as últimas camadas da MobileNetV2 reconhecem formas mais específicas, e ajustá-las ajuda a separar dois Pokémon da mesma cor.

Repetindo com taxa 1e-3 (`experimentos/lr_alta`), a loss de validação pulou de 0,25 para 8,05 já na primeira época de fine-tuning e chegou a 26,9 na época 11, quando a acurácia de validação caiu para 27%. Enquanto isso, a acurácia de treino subia até 99,4%. O EarlyStopping devolveu os pesos da época 14 (92,73%), mas mesmo esse modelo errou mais o fundo: recall de 77% na validação e 79% dos frames de fundo do vídeo de teste.

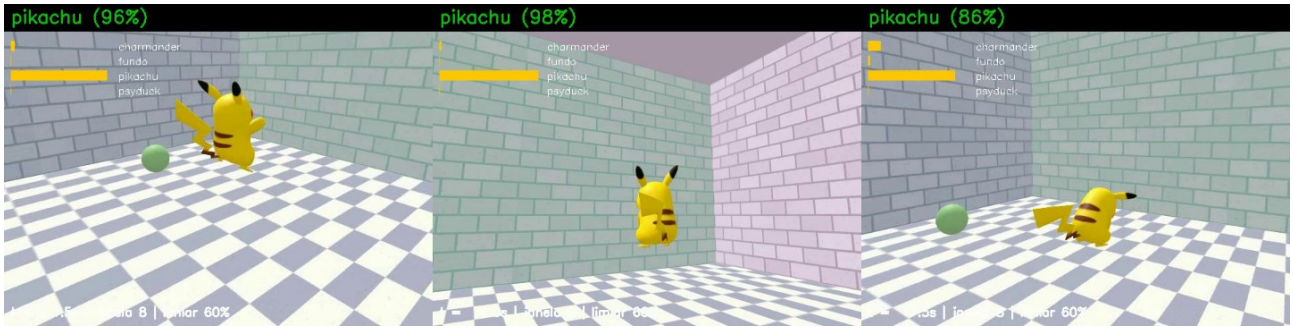
A taxa precisa ser baixa porque os pesos da base já são bons, aprendidos com mais de 1 milhão de imagens da ImageNet. Com 1e-3, cada passo muda muito esses pesos, o modelo perde os filtros gerais e se adapta demais às nossas 1.284 fotos de treino. Com 1e-5 os pesos mudam aos poucos e o modelo só refina o que já sabia.



Curvas com fine-tuning em lr 1e-3: a loss de validação dispara a partir da época 8.

3. O modelo reconhece o Pokémon de qualquer ângulo?

Usamos o ângulo da câmera que o gerador salva em cada frame para tirar do treino todos os frames do Pikachu com a câmera entre 120° e 240°, ou seja, de costas: foram 135 de 408 fotos (`experimentos/sem_costas.py`). Depois rodamos o `detectar.py` em costas_pikachu.mp4, um vídeo só das costas, na sala de teste. O modelo acertou: disse pikachu nos 120 frames, com confiança média de 93,9%. O modelo completo, que viu as costas, ficou com 98,8%. Tirar as costas baixou a confiança, mas não a ponto de errar, e a probabilidade de charmander subiu de 0,8% para 4,2%.



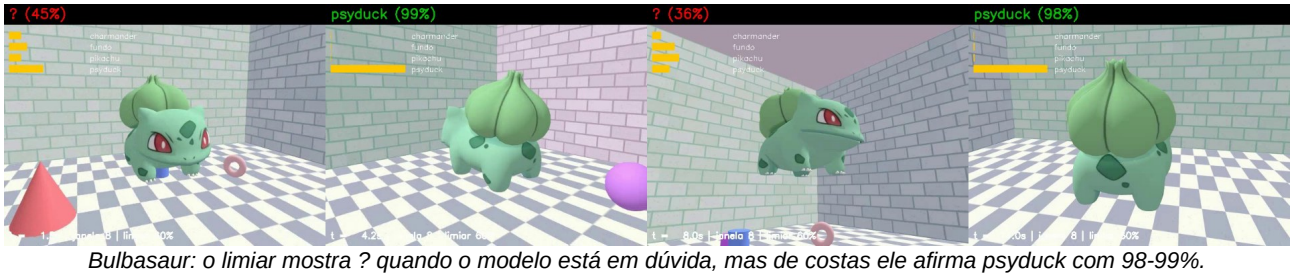
Modelo treinado sem as costas do Pikachu, rodando em costas_pikachu.mp4.

Pelos resultados, o modelo usa principalmente a cor e partes que aparecem de vários lados. De costas o Pikachu ainda mostra o amarelo, as orelhas compridas com ponta preta, as listras marrons e a cauda em zigue-zague, e as orelhas e a cauda também aparecem nas vistas de lado e de três quartos que ficaram no treino. A cor pesa bastante: quase todos os erros do Pikachu na validação foram para o Psyduck, o outro amarelo.

Duas estratégias para ângulos novos: (1) cobrir mais ângulos no treino, com passos menores de rotação e mais elevações, e usar data augmentation que apaga um retângulo aleatório da imagem, para o modelo não depender de uma parte só; (2) diminuir a dependência da cor, com augmentation de matiz e saturação ou parte das fotos em tons de cinza, para o modelo prestar mais atenção na forma, e misturar fontes diferentes do mesmo Pokémon, como carta, pelúcia e desenho.

4. O que acontece com um Pokémon que o modelo não conhece?

Mostramos o Bulbasaur, que não está no treino (desconhecido_bulbasaur.mp4). O modelo não disse que não conhecia. Olhando frame por frame, chamou de psyduck 67% dos frames (97 de 144), de fundo 17%, charmander 10% e pikachu 6%, com confiança média de 70%. Com a suavização e o limiar de 60%, 55% dos frames viraram ?, mas 41% continuaram como psyduck, às vezes com 98% ou 99% de confiança, principalmente de costas, quando o bulbo verde e redondo lembra o corpo do Psyduck. A linha do tempo listou psyduck de 2,9 a 5,6 s e de 10,2 a 12,0 s.



Bulbasaur: o limiar mostra ? quando o modelo está em dúvida, mas de costas ele afirma psyduck com 98-99%.

Isso acontece porque a softmax transforma as saídas da última camada em probabilidades que sempre somam 100%, divididas só entre as 4 classes conhecidas. Não existe uma saída "nenhuma dessas": a classe com a saída mais alta leva a maior fatia, mesmo que nenhuma combine bem. O limiar ajuda quando o modelo fica em dúvida, como em metade deste vídeo, mas não resolve quando ele erra com confiança. A classe fundo também não resolve, porque ela aprendeu "nenhum objeto no centro da imagem", e o Bulbasaur é claramente um objeto. Por isso o fundo ficou com só 19,6% de probabilidade média. Uma solução seria criar uma classe "outro Pokémon" com vários Pokémon diferentes, o que seria fácil de gerar, já que a API tem mais de mil modelos.

5. Classificação ou detecção?

Gravamos o Pikachu e o Charmander juntos (dois_pokemon.mp4). O modelo sempre escolheu um só: charmander em 76% dos frames e pikachu em 24%. Na linha do tempo, o rótulo ficou em charmander e trocou para pikachu em dois trechos curtos (1,6 a 2,4 s e 8,2 a 9,1 s), quando o Pikachu estava mais perto da câmera ou na frente do Charmander. A confiança continuou alta (mediana de 98%), então o modelo nem dá sinal de que há dois Pokémon.



Dois Pokémon no quadro: o rótulo é sempre um só e troca conforme o que está em destaque.

É o esperado, porque fizemos classificação de imagens: o modelo recebe a imagem inteira e devolve uma classe, com probabilidades que somam 100%. Ele responde o que tem na imagem, mas não onde. A detecção de objetos, como no YOLO, responde as duas coisas: para cada objeto devolve uma caixa (posição, largura e altura), a classe e a confiança, e pode devolver várias caixas na mesma imagem.

Para isso mudariam os dados e o modelo. Nos dados, cada foto precisaria de anotação: uma caixa em volta de cada Pokémon com o nome dele, feita em ferramentas como o LabelImg ou o Roboflow, e fotos com vários Pokémon juntos. No nosso caso daria para gerar as caixas automaticamente, projetando na imagem o contorno 3D de cada modelo. No modelo, a cabeça de classificação (GlobalAveragePooling e Dense com softmax) seria trocada por uma cabeça de detecção, que prevê caixas e classes em várias posições da imagem, treinada com uma perda que soma o erro da posição da caixa e o erro da classe. A classe fundo deixaria de existir, porque nenhuma caixa já significa que não há Pokémon.